

Introduction to Optimization
Spring 2026
University of Science and Technology of China

Lecturer: Nick

Posted: Mar. 23rd, 2025

Name: Jinghao Chen

Homework 2

Due: Apr. 6th, 2025

ID: PB23010359

Exercise 1:

For all possible cases of $P \in \mathbb{S}^d$, $q \in \mathbb{R}^d$:

$$\text{minimize} \quad x^T P x + q^T x$$

Solution 1:

Let

$$f(x) = x^T P x + q^T x, \quad P \in \mathbb{S}^d, \quad q \in \mathbb{R}^d.$$

Since $P = P^T$, we classify all cases by the sign of P .

$$\nabla f(x) = 2Px + q.$$

① **If $P \not\succeq 0$.**

Then $\exists v \neq 0$ such that

$$v^T P v < 0.$$

Take $x = tv$. Then

$$f(tv) = t^2 v^T P v + tq^T v.$$

Hence

$$t \rightarrow \infty \implies f(tv) \rightarrow -\infty.$$

Therefore,

$$\inf_{x \in \mathbb{R}^d} f(x) = -\infty.$$

② **If $P \succeq 0$ and $q \notin \text{Range}(P)$.**

Since $P = P^T$,

$$\text{Range}(P) = \text{Null}(P)^\perp.$$

Thus

$$q \notin \text{Range}(P) \implies \exists z \in \text{Null}(P) \text{ such that } q^T z \neq 0.$$

Because $Pz = 0$,

$$f(x + tz) = (x + tz)^T P(x + tz) + q^T(x + tz) = x^T P x + q^T x + tq^T z.$$

Choose the sign of t so that $tq^T z \rightarrow -\infty$. Then

$$\inf_{x \in \mathbb{R}^d} f(x) = -\infty.$$

③ **If $P \succeq 0$ and $q \in \text{Range}(P)$.**

Then

$$2Px + q = 0$$

has a solution. Let x_0 satisfy

$$2Px_0 + q = 0.$$

Then

$$q = -2Px_0,$$

and for any $x \in \mathbb{R}^d$,

$$\begin{aligned}
 f(x) - f(x_0) &= x^T P x + q^T x - (x_0^T P x_0 + q^T x_0) \\
 &= x^T P x - 2x_0^T P x - x_0^T P x_0 + 2x_0^T P x_0 \\
 &= x^T P x - 2x_0^T P x + x_0^T P x_0 \\
 &= (x - x_0)^T P (x - x_0) \geq 0.
 \end{aligned}$$

Hence x_0 is a minimizer.

Moreover,

$$2Px + q = 0 \iff P(x - x_0) = 0 \iff x - x_0 \in \text{Null}(P).$$

So the set of minimizers is

$$\arg \min f = \{x : 2Px + q = 0\} = x_0 + \text{Null}(P).$$

If $P^\dagger$ is the Moore–Penrose pseudoinverse, one minimizer is

$$x^\star = -\frac{1}{2}P^\dagger q,$$

and

$$\arg \min f = -\frac{1}{2}P^\dagger q + \text{Null}(P).$$

The optimal value is

$$f^\star = -\frac{1}{4}q^T P^\dagger q.$$

Therefore,

If $P \not\succeq 0$, $\inf f(x) = -\infty$. If $P \succeq 0$, $q \notin \text{Range}(P)$, $\inf f(x) = -\infty$. If $P \succeq 0$, $q \in \text{Range}(P)$, $\arg \min f = \{x : 2Px + q = 0\}$.

In particular, if $P \succ 0$, then

$$x^\star = -\frac{1}{2}P^{-1}q, \quad f^\star = -\frac{1}{4}q^T P^{-1}q,$$

and the minimizer is unique.

If $P = 0$, then

$$f(x) = q^T x.$$

Thus

$$q = 0 \implies \text{every } x \text{ is optimal}, \quad q \neq 0 \implies \inf f(x) = -\infty.$$

■

Exercise 2:

Some simple LPs. Give an explicit solution of each of the following LPs.

- (a) *Minimizing a linear function over an affine set.*

$$\begin{array}{ll}\text{minimize} & c^T x \\ \text{subject to} & Ax = b.\end{array}$$

- (b) *Minimizing a linear function over a halfspace.*

$$\begin{array}{ll}\text{minimize} & c^T x \\ \text{subject to} & a^T x \leq b,\end{array}$$

where $a \neq 0$.

- (c) *Minimizing a linear function over a rectangle.*

$$\begin{array}{ll}\text{minimize} & c^T x \\ \text{subject to} & l \preceq x \preceq u,\end{array}$$

where l and u satisfy $l \preceq u$.

- (d) *Minimizing a linear function over the probability simplex.*

$$\begin{array}{ll}\text{minimize} & c^T x \\ \text{subject to} & \mathbf{1}^T x = 1, \quad x \succeq 0.\end{array}$$

What happens if the equality constraint is replaced by an inequality $\mathbf{1}^T x \leq 1$?

We can interpret this LP as a simple portfolio optimization problem. The vector x represents the allocation of our total budget over different assets, with x_i the fraction invested in asset i . The return of each investment is fixed and given by $-c_i$, so our total return (which we want to maximize) is $-c^T x$. If we replace the budget constraint $\mathbf{1}^T x = 1$ with an inequality $\mathbf{1}^T x \leq 1$, we have the option of not investing a portion of the total budget.

Solution 2:

(a) Let $F = \{x : Ax = b\}$.

① **If** $b \notin \text{Range}(A)$.

Then $F = \emptyset$. Hence the LP is infeasible.

② **If** $b \in \text{Range}(A)$ **and** $c \in \text{Range}(A^T)$.

Take any x_0 such that

$$Ax_0 = b.$$

Then every feasible point can be written as

$$x = x_0 + z, \quad z \in \text{Null}(A).$$

Since $c \in \text{Range}(A^T) = \text{Null}(A)^\perp$,

$$c^T z = 0, \quad \forall z \in \text{Null}(A).$$

Therefore,

$$c^T x = c^T x_0, \quad \forall x \in F.$$

So every feasible point is optimal:

$$\arg \min c^T x = \{x : Ax = b\}.$$

③ **If** $b \in \text{Range}(A)$ **and** $c \notin \text{Range}(A^T)$.

Then

$$c \notin \text{Null}(A)^\perp \implies \exists z \in \text{Null}(A) \text{ such that } c^T z \neq 0.$$

Take any feasible x_0 . Since $A(x_0 + tz) = b$,

$$c^T(x_0 + tz) = c^T x_0 + t c^T z.$$

Choose the sign of t so that $t c^T z \rightarrow -\infty$. Hence

$$\inf c^T x = -\infty.$$

(b) ① **If** $c \notin -\mathbb{R}_+ a$.

Then $\exists d$ such that

$$a^T d = 0, \quad c^T d < 0.$$

Take any feasible x_0 , i.e. $a^T x_0 \leq b$. Then

$$a^T(x_0 + td) = a^T x_0 \leq b, \quad \forall t \geq 0,$$

so $x_0 + td$ is feasible for all $t \geq 0$. But

$$c^T(x_0 + td) = c^T x_0 + t c^T d \rightarrow -\infty.$$

Hence

$$\inf c^T x = -\infty.$$

② If $c = -\lambda a$ for some $\lambda \geq 0$.

Then

$$c^T x = -\lambda a^T x.$$

If $\lambda = 0$, then $c = 0$, and every feasible point is optimal.

If $\lambda > 0$, then

$$a^T x \leq b \implies c^T x = -\lambda a^T x \geq -\lambda b.$$

Equality holds iff

$$a^T x = b.$$

Therefore,

$$p^* = -\lambda b, \quad \arg \min c^T x = \{x : a^T x = b\}.$$

(c) Since

$$c^T x = \sum_{i=1}^n c_i x_i, \quad l_i \leq x_i \leq u_i,$$

the problem separates coordinatewise.

For each i ,

$$x_i^* = \begin{cases} l_i, & c_i > 0, \\ u_i, & c_i < 0, \\ \text{any value in } [l_i, u_i], & c_i = 0. \end{cases}$$

Hence

$$\arg \min c^T x = \left\{ x : x_i = \begin{cases} l_i, & c_i > 0, \\ u_i, & c_i < 0, \\ \text{any } x_i \in [l_i, u_i], & c_i = 0 \end{cases} \right\}.$$

The optimal value is

$$p^* = \sum_{i=1}^n \begin{cases} c_i l_i, & c_i > 0, \\ c_i u_i, & c_i < 0, \\ 0, & c_i = 0. \end{cases}$$

(d) Let

$$m = \min_i c_i, \quad I = \{i : c_i = m\}.$$

If

$$\mathbf{1}^T x = 1, \quad x \succeq 0,$$

then

$$c^T x = \sum_{i=1}^n c_i x_i \geq \sum_{i=1}^n m x_i = m \sum_{i=1}^n x_i = m.$$

Equality holds iff

$$x_i = 0, \quad \forall i \notin I.$$

Therefore,

$$p^* = m,$$

and

$$\arg \min c^T x = \{x : \mathbf{1}^T x = 1, x \succeq 0, x_i = 0 \forall i \notin I\}.$$

In particular, any e_j with $j \in I$ is optimal.

Now replace $\mathbf{1}^T x = 1$ by

$$\mathbf{1}^T x \leq 1, \quad x \succeq 0.$$

① **If** $m < 0$.

Then it is optimal to invest the whole budget in the assets with smallest c_i :

$$p^* = m,$$

and

$$\arg \min c^T x = \{x : \mathbf{1}^T x = 1, x \succeq 0, x_i = 0 \forall i \notin I\}.$$

② **If** $m = 0$.

Then

$$c^T x \geq 0,$$

and equality holds iff

$$x_i = 0, \quad \forall i \notin I.$$

Hence

$$p^* = 0,$$

and

$$\arg \min c^T x = \{x : \mathbf{1}^T x \leq 1, x \succeq 0, x_i = 0 \forall i \notin I\}.$$

③ **If** $m > 0$.

Then for every $x \succeq 0$,

$$c^T x \geq 0,$$

and equality holds only at

$$x = 0.$$

Thus

$$p^* = 0, \quad x^* = 0.$$

■

Exercise 3:

Some simple QCQPs. Give an explicit solution of each of the following QCQPs.

- (a) *Minimizing a linear function over an ellipsoid centered at the origin.*

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && x^T A x \leq 1, \end{aligned}$$

where $A \in \mathbb{S}_{++}^n$ and $c \neq 0$. What is the solution if the problem is not convex ($A \notin \mathbb{S}_+^n$)?

- (b) *Minimizing a linear function over an ellipsoid.*

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && (x - x_c)^T A (x - x_c) \leq 1, \end{aligned}$$

where $A \in \mathbb{S}_{++}^n$ and $c \neq 0$.

- (c) *Minimizing a quadratic form over an ellipsoid centered at the origin.*

$$\begin{aligned} & \text{minimize} && x^T B x \\ & \text{subject to} && x^T A x \leq 1, \end{aligned}$$

where $A \in \mathbb{S}_{++}^n$ and $B \in \mathbb{S}_+^n$. Also consider the nonconvex extension with $B \notin \mathbb{S}_+^n$. (See §B.1.)

Reference B.1. Single constraint quadratic optimization.

We consider the problem with one constraint

$$\begin{aligned} & \text{minimize} && x^T A_0 x + 2b_0^T x + c_0 \\ & \text{subject to} && x^T A_1 x + 2b_1^T x + c_1 \leq 0, \end{aligned} \tag{B.1}$$

with variable $x \in \mathbb{R}^n$, and problem parameters $A_i \in \mathbb{S}^n$, $b_i \in \mathbb{R}^n$, $c_i \in \mathbb{R}$. We do not assume that $A_i \succeq 0$, so problem (B.1) is not a convex optimization problem.

Solution 3:

(a) Consider

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && x^T A x \leq 1, \end{aligned} \quad A \in \mathbb{S}_{++}^n, \ c \neq 0.$$

Since $A \succ 0$,

$$c^T x = (A^{-1/2}c)^T (A^{1/2}x) \geq -\|A^{-1/2}c\|_2 \|A^{1/2}x\|_2.$$

Also,

$$x^T A x \leq 1 \implies \|A^{1/2}x\|_2 \leq 1.$$

Hence

$$c^T x \geq -\|A^{-1/2}c\|_2 = -\sqrt{c^T A^{-1}c}.$$

Equality holds when

$$A^{1/2}x = -\frac{A^{-1/2}c}{\|A^{-1/2}c\|_2},$$

i.e.,

$$x^* = -\frac{A^{-1}c}{\sqrt{c^T A^{-1}c}}.$$

Therefore,

$$p^* = -\sqrt{c^T A^{-1}c}, \quad x^* = -\frac{A^{-1}c}{\sqrt{c^T A^{-1}c}}.$$

Now consider $A \notin \mathbb{S}_+^n$.Then $\exists v \neq 0$ such that

$$v^T A v < 0.$$

① If $c^T v \neq 0$.Choose the sign of v so that

$$c^T v < 0.$$

Then for any $t > 0$,

$$(tv)^T A (tv) = t^2 v^T A v \leq 1,$$

so tv is feasible, and

$$c^T (tv) = t c^T v \rightarrow -\infty.$$

Hence

$$\inf c^T x = -\infty.$$

② If $c^T v = 0$.

Let

$$d = -c + tv.$$

Then

$$c^T d = -\|c\|_2^2 < 0.$$

Moreover,

$$d^T A d = c^T A c - 2t c^T A v + t^2 v^T A v.$$

Since $v^T A v < 0$, for t large enough,

$$d^T A d < 0.$$

Hence for any $\alpha > 0$,

$$(\alpha d)^T A (\alpha d) = \alpha^2 d^T A d \leq 1,$$

so αd is feasible, and

$$c^T(\alpha d) = \alpha c^T d \rightarrow -\infty.$$

Thus

$$\inf c^T x = -\infty.$$

Therefore, if $A \notin \mathbb{S}_+^n$, the problem is unbounded below.

(b) Consider

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && (x - x_c)^T A (x - x_c) \leq 1, \end{aligned} \quad A \in \mathbb{S}_{++}^n, \quad c \neq 0.$$

Let

$$y = x - x_c.$$

Then the problem becomes

$$\begin{aligned} & \text{minimize} && c^T(y + x_c) = c^T y + c^T x_c \\ & \text{subject to} && y^T A y \leq 1. \end{aligned}$$

By part (a),

$$y^* = -\frac{A^{-1}c}{\sqrt{c^T A^{-1}c}}.$$

Hence

$$x^* = x_c - \frac{A^{-1}c}{\sqrt{c^T A^{-1}c}},$$

and

$$p^* = c^T x_c - \sqrt{c^T A^{-1}c}.$$

(c) Consider

$$\begin{aligned} & \text{minimize} && x^T B x \\ & \text{subject to} && x^T A x \leq 1, \end{aligned} \quad A \in \mathbb{S}_{++}^n.$$

Let

$$y = A^{1/2} x, \quad C = A^{-1/2} B A^{-1/2}.$$

Then

$$x = A^{-1/2} y,$$

and the problem becomes

$$\begin{aligned} & \text{minimize} && y^T C y \\ & \text{subject to} && \|y\|_2^2 \leq 1. \end{aligned}$$

Since $C \in \mathbb{S}^n$, let

$$\lambda_{\min}(C)$$

be its smallest eigenvalue.

For any $y = ru$ with $\|u\|_2 = 1$ and $0 \leq r \leq 1$,

$$y^T C y = r^2 u^T C u \geq r^2 \lambda_{\min}(C).$$

① **If $\lambda_{\min}(C) \geq 0$.**

Then

$$y^T C y \geq 0,$$

and equality is attained at

$$y = 0.$$

Hence

$$p^* = 0, \quad x^* = 0.$$

In particular, if $B \succeq 0$, then $C \succeq 0$, so

$$p^* = 0, \quad x^* = 0.$$

② **If $\lambda_{\min}(C) < 0$.**

Then $r = 1$ is optimal, and equality is attained when

$$Cu = \lambda_{\min}(C)u, \quad \|u\|_2 = 1.$$

So

$$y^* = u_{\min},$$

where $u_{\min}$ is any unit eigenvector associated with $\lambda_{\min}(C)$. Therefore,

$$x^* = A^{-1/2}u_{\min}, \quad p^* = \lambda_{\min}(C).$$

Equivalently,

$$p^* = \lambda_{\min}\left(A^{-1/2}BA^{-1/2}\right) = \min_{x \neq 0} \frac{x^T B x}{x^T A x},$$

and any optimizer satisfies

$$Bx^* = p^* Ax^*, \quad (x^*)^T A x^* = 1,$$

whenever $p^* < 0$.

In particular, if $B \notin \mathbb{S}_+^n$, then

$$\lambda_{\min}\left(A^{-1/2}BA^{-1/2}\right) < 0,$$

so the optimal value is

$$p^* = \lambda_{\min}\left(A^{-1/2}BA^{-1/2}\right),$$

and any optimal solution is a generalized eigenvector of

$$Bx = \lambda Ax$$

corresponding to the smallest generalized eigenvalue, normalized by

$$x^T Ax = 1.$$

■

Exercise 4:

Solve the following optimization problems in CVXPY. Your solution should include both the optimal value and the optimal decision variables, as well as the scripts you used to get them.

(a)

$$\begin{aligned} & \text{minimize} && x_1 + 2x_2 + 3x_3 - x_4 \\ & \text{subject to} && x_1 - x_2 + x_3 - 2x_4 \leq 6, \\ & && 2x_1 + x_2 - x_3 \geq 2, \\ & && -x_1 + x_2 - x_3 + x_4 \geq 8, \\ & && 0 \leq x_1 \leq 3, \\ & && 1 \leq x_2 \leq 4, \\ & && 0 \leq x_3 \leq 10, \\ & && 2 \leq x_4 \leq 5. \end{aligned}$$

(b)

$$\begin{aligned} & \text{minimize} && 9x_1^2 + 9x_2^2 - 30x_1 - 72x_2 \\ & \text{subject to} && -2x_1 - x_2 \geq -4, \\ & && x_1 \geq 0, \ x_2 \geq 0. \end{aligned}$$

(c)

$$\begin{aligned} & \text{minimize} && t \\ & \text{subject to} && \begin{bmatrix} 1 & 0 & 7 \\ 0 & 5 & 3 \\ 7 & 3 & 2 \end{bmatrix} + x_1 \begin{bmatrix} 2 & -2 & 2 \\ -2 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix} + x_2 \begin{bmatrix} 9 & 2 & 1 \\ 2 & 5 & 6 \\ 1 & 6 & 4 \end{bmatrix} \preceq tI. \end{aligned}$$

Solution 4:

We solve the following problems in CVXPY. The scripts are included from the `py` folder.

(a) Running `py/EX_4/4_1.py`, we obtain

$$p^* \approx 0.9999999990835633,$$

$$x^* \approx \begin{bmatrix} -6.87392759 \times 10^{-11} \\ 3.00000000 \\ -8.55986395 \times 10^{-11} \\ 5.00000000 \end{bmatrix}.$$

Hence, up to numerical precision,

$$x^* = \begin{bmatrix} 0 \\ 3 \\ 0 \\ 5 \end{bmatrix}, \quad p^* = 1.$$

Scripts:

```
import cvxpy as cp

x = cp.Variable(4)

objective = cp.Minimize(x[0] + 2*x[1] + 3*x[2] - x[3])
constraints = [
    x[0] - x[1] + x[2] - 2*x[3] <= 6,
    2*x[0] + x[1] - x[2] >= 2,
    -x[0] + x[1] - x[2] + x[3] >= 8,
    0 <= x[0], x[0] <= 3,
    1 <= x[1], x[1] <= 4,
    0 <= x[2], x[2] <= 10,
    2 <= x[3], x[3] <= 5
]

prob = cp.Problem(objective, constraints)
prob.solve()

print("optimal value =", prob.value)
print("optimal x =", x.value)
```

(b) Running `py/EX_4/4_2.py`, we obtain

$$p^* = -149.0,$$

$$x^* = \begin{bmatrix} 0.33333333 \\ 3.33333333 \end{bmatrix} \approx \begin{bmatrix} \frac{1}{3} \\ \frac{10}{3} \end{bmatrix}.$$

Therefore,

$$x^* = \begin{bmatrix} \frac{1}{3} \\ \frac{10}{3} \end{bmatrix}, \quad p^* = -149.$$

Scripts:

```
import cvxpy as cp

x = cp.Variable(2)

objective = cp.Minimize(9*cp.square(x[0]) + 9*cp.square(x[1]) - 30*
    x[0] - 72*x[1])
constraints = [
    -2*x[0] - x[1] >= -4,
    x[0] >= 0,
    x[1] >= 0
]

prob = cp.Problem(objective, constraints)
prob.solve()

print("optimal value =", prob.value)
print("optimal x =", x.value)
```

(c) Running py/EX_4/4_3.py, we obtain

$$p^* \approx -0.07471365166327809,$$

$$x^* \approx \begin{bmatrix} -2.78130964 \\ -1.04415091 \end{bmatrix},$$

and

$$t^* \approx -0.07471365166327809.$$

Therefore,

$$x^* \approx \begin{bmatrix} -2.78130964 \\ -1.04415091 \end{bmatrix}, \quad t^* \approx -0.07471365166327809.$$

Scripts:

```
import cvxpy as cp
import numpy as np

x = cp.Variable(2)
t = cp.Variable()

A0 = np.array([
    [1, 0, 7],
    [0, 5, 3],
    [7, 3, 2]
], dtype=float)

A1 = np.array([
```

```
[2, -2, 2],  
[-2, 1, 0],  
[2, 0, 1]  
], dtype=float)  
  
A2 = np.array([  
    [9, 2, 1],  
    [2, 5, 6],  
    [1, 6, 4]  
], dtype=float)  
  
M = A0 + x[0]*A1 + x[1]*A2  
  
objective = cp.Minimize(t)  
constraints = [  
    M <= t*np.eye(3)  
]  
  
prob = cp.Problem(objective, constraints)  
prob.solve(solver=cp.SCS, eps=1e-8)  
  
print("optimal value =", prob.value)  
print("optimal x =", x.value)  
print("optimal t =", t.value)
```



Exercise 5:

Linear regression. The California Housing dataset contains 20,640 instances with 8 attributes and 1 target. We provide two files for training and two for testing:

(a) ‘train_data.csv’ contains the attributes and ‘train_target.csv’ contains the targets for 16,640 instances (respectively denoted as $A \in \mathbb{R}^{16640 \times 8}$, $b \in \mathbb{R}^{16640}$);

(b) ‘test_data.csv’ and ‘test_target.csv’ contain the attributes and targets for the remaining 4,000 instances ($A_{\text{test}} \in \mathbb{R}^{4000 \times 8}$, $b_{\text{test}} \in \mathbb{R}^{4000}$). The goal is to obtain a model $x \in \mathbb{R}^8$ with low test error $\|A_{\text{test}}x - b_{\text{test}}\|_2^2$, using only the training data A, b .

Consider the following three alternatives:

- Least squares (LS):

$$\text{minimize } \|Ax - b\|_2^2$$

- LASSO ($\lambda > 0$):

$$\text{minimize } \|Ax - b\|_2^2 + \lambda \|x\|_1$$

- Ridge regression ($\lambda > 0$):

$$\text{minimize } \|Ax - b\|_2^2 + \lambda \|x\|_2^2$$

(a) Obtain the least squares solution x_{LS} .

(b) Solve LASSO for various choices of λ and plot

$$(\|Ax - b\|_2^2, \|x\|_1)$$

for each choice. What do you observe? Pick your “best” solution as x_{LASSO} .

(c) Repeat the same as in (b) to obtain x_{Ridge} .

(d) Evaluate $\|A_{\text{test}}x - b_{\text{test}}\|_2^2$ for the three solutions you obtained in (a), (b), (c), and discuss your findings (please share your plots and scripts).

Solution 5:

We solve the three models in CVXPY, using only the training data. For LASSO and Ridge, we choose λ by a validation split from the training set.

(a) The least squares solution is

$$x_{\text{LS}} \approx \begin{bmatrix} 0.510808701 \\ 0.0156527654 \\ -0.180357769 \\ 0.856848720 \\ 0.00000700193183 \\ -0.00516596461 \\ -0.0665626420 \\ -0.0172469203 \end{bmatrix}.$$

The corresponding errors are

$$\|Ax_{\text{LS}} - b\|_2^2 \approx 10151.831797126635,$$

$$\|A_{\text{test}}x_{\text{LS}} - b_{\text{test}}\|_2^2 \approx 2320.060012898806.$$

Scripts:

```
from pathlib import Path
import numpy as np
import cvxpy as cp

def load_csv_matrix(path: Path) -> np.ndarray:
    try:
        return np.loadtxt(path, delimiter=",")
    except ValueError:
        return np.loadtxt(path, delimiter=",", skiprows=1)

def load_csv_vector(path: Path) -> np.ndarray:
    try:
        arr = np.loadtxt(path, delimiter=",")
    except ValueError:
        arr = np.loadtxt(path, delimiter=",", skiprows=1)
    return np.asarray(arr).reshape(-1)

def main() -> None:
    base_dir = Path(__file__).resolve().parent
    data_dir = base_dir / "cal_housing"
    results_dir = base_dir / "results"
    results_dir.mkdir(parents=True, exist_ok=True)

    A = load_csv_matrix(data_dir / "train_data.csv")
    b = load_csv_vector(data_dir / "train_target.csv")
```

```

A_test = load_csv_matrix(data_dir / "test_data.csv")
b_test = load_csv_vector(data_dir / "test_target.csv")

x = cp.Variable(A.shape[1])

objective = cp.Minimize(cp.sum_squares(A @ x - b))
prob = cp.Problem(objective)
prob.solve()

x_star = np.asarray(x.value).reshape(-1)
train_error = float(np.linalg.norm(A @ x_star - b) ** 2)
test_error = float(np.linalg.norm(A_test @ x_star - b_test) **
2)

np.savetxt(results_dir / "5_1_x_ls.csv", x_star, delimiter=",")

with open(results_dir / "5_1_results.txt", "w", encoding="utf-8") as f:
    f.write(f"train_error = {train_error}\n")
    f.write(f"test_error = {test_error}\n")
    f.write("x_LS =\n")
    for val in x_star:
        f.write(f"{val}\n")

print("train_error =", train_error)
print("test_error =", test_error)
print("x_LS =", x_star)

if __name__ == "__main__":
    main()

```

(b) For LASSO, we solve

$$\min_x \|Ax - b\|_2^2 + \lambda \|x\|_1$$

for various choices of λ , and plot

$$(\|Ax - b\|_2^2, \|x\|_1).$$

We observe a trade-off curve: as λ increases, $\|x\|_1$ decreases, while the training residual $\|Ax - b\|_2^2$ increases. The solution also becomes sparser.

Our selected parameter is

$$\lambda_{\text{LASSO}} \approx 67.68750009458527,$$

and the corresponding solution is

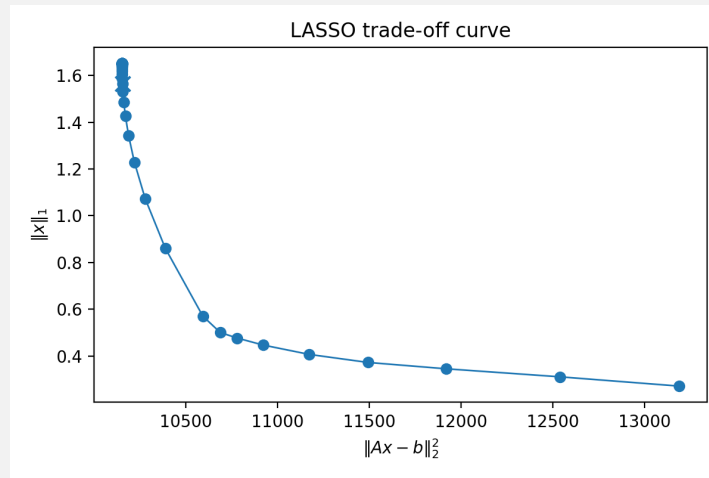
$$x_{\text{LASSO}} \approx \begin{bmatrix} 0.502738556 \\ 0.0155185722 \\ -0.166612308 \\ 0.789670245 \\ 0.00000510888522 \\ -0.00492432611 \\ -0.0668538213 \\ -0.0176348723 \end{bmatrix}.$$

Moreover,

$$\|x_{\text{LASSO}}\|_1 \approx 1.563957808927983,$$

$$\|Ax_{\text{LASSO}} - b\|_2^2 \approx 10154.882933608396,$$

$$\|A_{\text{test}}x_{\text{LASSO}} - b_{\text{test}}\|_2^2 \approx 2321.052665182842.$$



Scripts:

```
from pathlib import Path
import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

SEED = 0
VAL_RATIO = 0.2
LAMBDA_S = np.logspace(-4, 4, 60)

def load_csv_matrix(path: Path) -> np.ndarray:
    try:
        return np.loadtxt(path, delimiter=",")
    except ValueError:
```

```
        return np.loadtxt(path, delimiter=",", skiprows=1)

def load_csv_vector(path: Path) -> np.ndarray:
    try:
        arr = np.loadtxt(path, delimiter=",")
    except ValueError:
        arr = np.loadtxt(path, delimiter=",", skiprows=1)
    return np.asarray(arr).reshape(-1)

def train_val_split(A: np.ndarray, b: np.ndarray, val_ratio: float,
                    seed: int):
    n = A.shape[0]
    rng = np.random.default_rng(seed)
    perm = rng.permutation(n)
    n_val = int(round(n * val_ratio))
    val_idx = perm[:n_val]
    tr_idx = perm[n_val:]
    return A[tr_idx], b[tr_idx], A[val_idx], b[val_idx]

def compute_scale(A: np.ndarray, eps: float = 1e-12) -> np.ndarray:
    scale = A.std(axis=0, ddof=0)
    scale[scale < eps] = 1.0
    return scale

def fit_lasso(A: np.ndarray, b: np.ndarray, lam: float) -> np.
ndarray:
    x = cp.Variable(A.shape[1])
    objective = cp.Minimize(cp.sum_squares(A @ x - b) + lam * cp.
        norm1(x))
    prob = cp.Problem(objective)
    prob.solve(solver=cp.SCS, eps=1e-6, verbose=False)
    return np.asarray(x.value).reshape(-1)

def main() -> None:
    base_dir = Path(__file__).resolve().parent
    data_dir = base_dir / "cal_housing"
    results_dir = base_dir / "results"
    results_dir.mkdir(parents=True, exist_ok=True)

    A = load_csv_matrix(data_dir / "train_data.csv")
    b = load_csv_vector(data_dir / "train_target.csv")
    A_test = load_csv_matrix(data_dir / "test_data.csv")
    b_test = load_csv_vector(data_dir / "test_target.csv")

    A_tr, b_tr, A_val, b_val = train_val_split(A, b, VAL_RATIO,
        SEED)
    scale_tr = compute_scale(A_tr)
    A_tr_std = A_tr / scale_tr
```

```

val_errors = []
for lam in LAMBDA_S:
    z = fit_lasso(A_tr_std, b_tr, float(lam))
    x_val = z / scale_tr
    val_error = float(np.linalg.norm(A_val @ x_val - b_val) **
        2)
    val_errors.append(val_error)

best_idx = int(np.argmin(val_errors))
best_lambda = float(LAMBDA_S[best_idx])

scale_full = compute_scale(A)
A_std = A / scale_full
z_star = fit_lasso(A_std, b, best_lambda)
x_star = z_star / scale_full

train_error = float(np.linalg.norm(A @ x_star - b) ** 2)
test_error = float(np.linalg.norm(A_test @ x_star - b_test) **
    2)
l1_norm = float(np.linalg.norm(x_star, 1))

curve_train_errors = []
curve_l1_norms = []
curve_val_errors = []
for lam in LAMBDA_S:
    z = fit_lasso(A_std, b, float(lam))
    x_model = z / scale_full
    curve_train_errors.append(float(np.linalg.norm(A @ x_model
        - b) ** 2))
    curve_l1_norms.append(float(np.linalg.norm(x_model, 1)))
    curve_val_errors.append(float(np.linalg.norm(A_val @ (
        fit_lasso(A_tr_std, b_tr, float(lam)) / scale_tr) -
        b_val) ** 2))

curve = np.column_stack([LAMBDA_S, curve_train_errors,
    curve_l1_norms, curve_val_errors])
np.savetxt(
    results_dir / "5_2_curve.csv",
    curve,
    delimiter=",",
    header="lambda,train_error,l1_norm,val_error",
    comments=""
)
np.savetxt(results_dir / "5_2_x_lasso.csv", x_star, delimiter="
,")

plt.figure(figsize=(6, 4))
plt.plot(curve_train_errors, curve_l1_norms, marker="o",
    linewidth=1)
plt.scatter([train_error], [l1_norm], marker="x", s=80)
plt.xlabel(r"$\|Ax-b\|_2^2$")
plt.ylabel(r"$\|x\|_1$")

```

```

plt.title("LASSO trade-off curve")
plt.tight_layout()
plt.savefig(results_dir / "5_2_lasso_tradeoff.png", dpi=200)
plt.close()

with open(results_dir / "5_2_results.txt", "w", encoding="utf-8") as f:
    f.write(f"best_lambda = {best_lambda}\n")
    f.write(f"train_error = {train_error}\n")
    f.write(f"test_error = {test_error}\n")
    f.write(f"l1_norm = {l1_norm}\n")
    f.write(f"x_LASSO =\n")
    for val in x_star:
        f.write(f"{val}\n")

print("best_lambda =", best_lambda)
print("train_error =", train_error)
print("test_error =", test_error)
print("l1_norm =", l1_norm)
print("x_LASSO =", x_star)

if __name__ == "__main__":
    main()

```

(c) For Ridge regression, we solve

$$\min_x \|Ax - b\|_2^2 + \lambda \|x\|_2^2$$

for various choices of λ , and plot

$$(\|Ax - b\|_2^2, \|x\|_2^2).$$

We again observe a trade-off curve: as λ increases, $\|x\|_2^2$ decreases, while the training residual $\|Ax - b\|_2^2$ increases. Compared with LASSO, Ridge shrinks the coefficients more smoothly.

Our selected parameter is

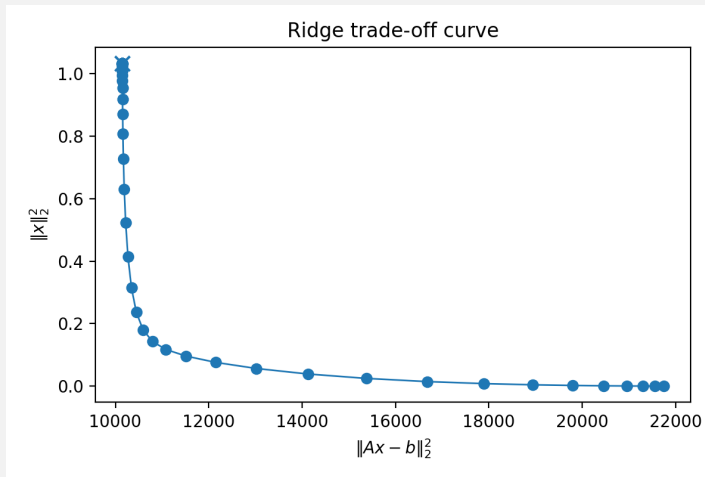
$$\lambda_{\text{Ridge}} = 0.0001,$$

and the corresponding solution is

$$x_{\text{Ridge}} \approx \begin{bmatrix} 0.510808686 \\ 0.0156527654 \\ -0.180357746 \\ 0.856848609 \\ 0.00000700192880 \\ -0.00516596448 \\ -0.0665626455 \\ -0.0172469218 \end{bmatrix}.$$

Also,

$$\begin{aligned}\|x_{\text{Ridge}}\|_2^2 &\approx 1.0326437068557672, \\ \|Ax_{\text{Ridge}} - b\|_2^2 &\approx 10151.83179712664, \\ \|A_{\text{test}}x_{\text{Ridge}} - b_{\text{test}}\|_2^2 &\approx 2320.060015006893.\end{aligned}$$



Scripts:

```
from pathlib import Path
import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

SEED = 0
VAL_RATIO = 0.2
LAMBDA_S = np.logspace(-4, 6, 60)

def load_csv_matrix(path: Path) -> np.ndarray:
    try:
        return np.loadtxt(path, delimiter=",")
    except ValueError:
        return np.loadtxt(path, delimiter=",", skiprows=1)

def load_csv_vector(path: Path) -> np.ndarray:
    try:
        arr = np.loadtxt(path, delimiter=",")
    except ValueError:
        arr = np.loadtxt(path, delimiter=",", skiprows=1)
    return np.asarray(arr).reshape(-1)

def train_val_split(A: np.ndarray, b: np.ndarray, val_ratio: float,
                    seed: int):
```

```
n = A.shape[0]
rng = np.random.default_rng(seed)
perm = rng.permutation(n)
n_val = int(round(n * val_ratio))
val_idx = perm[:n_val]
tr_idx = perm[n_val:]
return A[tr_idx], b[tr_idx], A[val_idx], b[val_idx]

def compute_scale(A: np.ndarray, eps: float = 1e-12) -> np.ndarray:
    scale = A.std(axis=0, ddof=0)
    scale[scale < eps] = 1.0
    return scale

def fit_ridge(A: np.ndarray, b: np.ndarray, lam: float) -> np.
ndarray:
    x = cp.Variable(A.shape[1])
    objective = cp.Minimize(cp.sum_squares(A @ x - b) + lam * cp.
        sum_squares(x))
    prob = cp.Problem(objective)
    prob.solve(solver=cp.SCS, eps=1e-7, verbose=False)
    return np.asarray(x.value).reshape(-1)

def main() -> None:
    base_dir = Path(__file__).resolve().parent
    data_dir = base_dir / "cal_housing"
    results_dir = base_dir / "results"
    results_dir.mkdir(parents=True, exist_ok=True)

    A = load_csv_matrix(data_dir / "train_data.csv")
    b = load_csv_vector(data_dir / "train_target.csv")
    A_test = load_csv_matrix(data_dir / "test_data.csv")
    b_test = load_csv_vector(data_dir / "test_target.csv")

    A_tr, b_tr, A_val, b_val = train_val_split(A, b, VAL_RATIO,
        SEED)
    scale_tr = compute_scale(A_tr)
    A_tr_std = A_tr / scale_tr

    val_errors = []
    for lam in LAMBDAS:
        z = fit_ridge(A_tr_std, b_tr, float(lam))
        x_val = z / scale_tr
        val_error = float(np.linalg.norm(A_val @ x_val - b_val) **
            2)
        val_errors.append(val_error)

    best_idx = int(np.argmin(val_errors))
    best_lambda = float(LAMBDAS[best_idx])

    scale_full = compute_scale(A)
```

```

A_std = A / scale_full
z_star = fit_ridge(A_std, b, best_lambda)
x_star = z_star / scale_full

train_error = float(np.linalg.norm(A @ x_star - b) ** 2)
test_error = float(np.linalg.norm(A_test @ x_star - b_test) **
2)
l2_norm_sq = float(np.linalg.norm(x_star, 2) ** 2)

curve_train_errors = []
curve_l2_norm_sq = []
curve_val_errors = []
for lam in LAMBDA_S:
    z = fit_ridge(A_std, b, float(lam))
    x_model = z / scale_full
    curve_train_errors.append(float(np.linalg.norm(A @ x_model
- b) ** 2))
    curve_l2_norm_sq.append(float(np.linalg.norm(x_model, 2) **
2))
    curve_val_errors.append(float(np.linalg.norm(A_val @ (
fit_ridge(A_tr_std, b_tr, float(lam)) / scale_tr) -
b_val) ** 2))

curve = np.column_stack([LAMBDA_S, curve_train_errors,
curve_l2_norm_sq, curve_val_errors])
np.savetxt(
    results_dir / "5_3_curve.csv",
    curve,
    delimiter=",",
    header="lambda,train_error,l2_norm_sq,val_error",
    comments=""
)
np.savetxt(results_dir / "5_3_x_ridge.csv", x_star, delimiter="
,")

plt.figure(figsize=(6, 4))
plt.plot(curve_train_errors, curve_l2_norm_sq, marker="o",
linewidth=1)
plt.scatter([train_error], [l2_norm_sq], marker="x", s=80)
plt.xlabel(r"$\|Ax-b\|_2^2$")
plt.ylabel(r"$\|x\|_2^2$")
plt.title("Ridge trade-off curve")
plt.tight_layout()
plt.savefig(results_dir / "5_3_ridge_tradeoff.png", dpi=200)
plt.close()

with open(results_dir / "5_3_results.txt", "w", encoding="utf-8
") as f:
    f.write(f"best_lambda = {best_lambda}\n")
    f.write(f"train_error = {train_error}\n")
    f.write(f"test_error = {test_error}\n")
    f.write(f"l2_norm_sq = {l2_norm_sq}\n")
    f.write("x_Ridge =\n")

```

```
for val in x_star:
    f.write(f"{val}\n")

print("best_lambda =", best_lambda)
print("train_error =", train_error)
print("test_error =", test_error)
print("l2_norm_sq =", l2_norm_sq)
print("x_Ridge =", x_star)

if __name__ == "__main__":
    main()
```

(d) The test errors of the three models are

$$\|A_{\text{test}}x_{\text{LS}} - b_{\text{test}}\|_2^2 \approx 2320.060012898806,$$

$$\|A_{\text{test}}x_{\text{LASSO}} - b_{\text{test}}\|_2^2 \approx 2321.052665182842,$$

$$\|A_{\text{test}}x_{\text{Ridge}} - b_{\text{test}}\|_2^2 \approx 2320.060015006893.$$

Hence,

$$\|A_{\text{test}}x_{\text{LS}} - b_{\text{test}}\|_2^2 < \|A_{\text{test}}x_{\text{Ridge}} - b_{\text{test}}\|_2^2 < \|A_{\text{test}}x_{\text{LASSO}} - b_{\text{test}}\|_2^2.$$

Therefore, among the three models, the least squares solution gives the smallest test error. Ridge regression is almost identical to least squares, since the selected λ is very small. LASSO yields a sparser solution, but its test error is slightly larger in this experiment.

■

Exercise 6:

The following problem is for *extra credit*.

Consider the problem of minimizing a convex differentiable function on

$$\{x \in \mathbb{R}^n \mid x \succeq 0, \mathbf{1}^T x = 1\}.$$

- (a) Show that the necessary and sufficient condition for optimality is

$$\left\{ x_i > 0 \implies \frac{\partial f}{\partial x_i} = \min_{j=1,\dots,n} \frac{\partial f}{\partial x_j} \right\} \cap \{x \succeq 0, \mathbf{1}^T x = 1\}.$$

- (b) For the case

$$f(x) := \frac{1}{2} \|x - y\|_2^2$$

(where $y \in \mathbb{R}^n$ is given) derive an efficient algorithm for computing the optimal solution and analyze its complexity.

- (c) For a randomly chosen $y \in \mathbb{R}^{500}$, solve the problem using your algorithm as well as via CVXPY. Compare and explain the implications (please share your results and scripts).

Solution 6:

Let

$$\Delta := \{x \in \mathbb{R}^n \mid x \succeq 0, \mathbf{1}^T x = 1\}.$$

(a) Consider

$$\min_{x \in \Delta} f(x),$$

where f is convex and differentiable.

The Lagrangian is

$$L(x, \nu, \mu) = f(x) + \nu(\mathbf{1}^T x - 1) - \mu^T x,$$

where

$$\mu \succeq 0.$$

The KKT conditions are

$$x \succeq 0, \quad \mathbf{1}^T x = 1, \quad \mu \succeq 0,$$

$$\nabla f(x) + \nu \mathbf{1} - \mu = 0,$$

$$\mu_i x_i = 0, \quad i = 1, \dots, n.$$

Since the problem is convex, these conditions are necessary and sufficient.

From

$$\nabla f(x) + \nu \mathbf{1} - \mu = 0,$$

we have

$$\mu_i = \frac{\partial f}{\partial x_i} + \nu.$$

Since $\mu_i \geq 0$,

$$\frac{\partial f}{\partial x_i} \geq -\nu, \quad i = 1, \dots, n.$$

If $x_i > 0$, then by complementary slackness,

$$\mu_i = 0,$$

so

$$\frac{\partial f}{\partial x_i} = -\nu.$$

Hence

$$x_i > 0 \implies \frac{\partial f}{\partial x_i} = -\nu.$$

Since $\mathbf{1}^T x = 1$, at least one component is positive, so

$$-\nu = \min_{j=1, \dots, n} \frac{\partial f}{\partial x_j}.$$

Therefore the necessary and sufficient optimality condition is

$$\left\{ x_i > 0 \implies \frac{\partial f}{\partial x_i} = \min_{j=1, \dots, n} \frac{\partial f}{\partial x_j} \right\} \cap \{x \succeq 0, \mathbf{1}^T x = 1\}.$$

(b) Now let

$$f(x) = \frac{1}{2} \|x - y\|_2^2.$$

Then

$$\frac{\partial f}{\partial x_i} = x_i - y_i.$$

By part (a), at the optimum,

$$x_i > 0 \implies x_i - y_i = \min_j (x_j - y_j).$$

Equivalently, there exists $\theta \in \mathbb{R}$ such that

$$x_i > 0 \implies x_i = y_i - \theta.$$

Together with $x_i \geq 0$, this gives

$$x_i^* = \max\{y_i - \theta, 0\}, \quad i = 1, \dots, n,$$

where θ is chosen so that

$$\sum_{i=1}^n \max\{y_i - \theta, 0\} = 1.$$

To compute θ , sort y in decreasing order:

$$u_1 \geq u_2 \geq \dots \geq u_n.$$

Define

$$\rho = \max \left\{ k \in \{1, \dots, n\} \mid u_k - \frac{1}{k} \left(\sum_{i=1}^k u_i - 1 \right) > 0 \right\},$$

and

$$\theta = \frac{1}{\rho} \left(\sum_{i=1}^{\rho} u_i - 1 \right).$$

Then

$$x_i^* = \max\{y_i - \theta, 0\}, \quad i = 1, \dots, n.$$

The algorithm requires:

$$\text{sorting: } O(n \log n), \quad \text{prefix sums: } O(n), \quad \text{thresholding: } O(n).$$

Hence the total complexity is

$$O(n \log n),$$

and the memory complexity is

$$O(n).$$

(c) We generated a random vector

$$y \in \mathbb{R}^{500}$$

using `np.random.default_rng(0).standard_normal(500)`.

Using the custom projection algorithm, we obtained

$$\theta = 2.458089440898643,$$

$$\text{algorithm_time} = 0.0009610999841243029,$$

$$\frac{1}{2}\|x_{\text{alg}} - y\|_2^2 = 254.33505541902846.$$

Using CVXPY, we obtained

$$\text{cvxpy_time} = 0.03362550004385412,$$

$$\frac{1}{2}\|x_{\text{cvx}} - y\|_2^2 = 254.3350554190285.$$

The two solutions agree up to numerical precision:

$$\|x_{\text{alg}} - x_{\text{cvx}}\|_2 = 8.416972462747517 \times 10^{-15},$$

$$\|x_{\text{alg}} - x_{\text{cvx}}\|_\infty = 4.884981308350689 \times 10^{-15}.$$

They also satisfy the simplex constraints numerically:

$$\mathbf{1}^T x_{\text{alg}} = 0.9999999999999996, \quad \mathbf{1}^T x_{\text{cvx}} = 0.99999999999999796,$$

$$\min_i (x_{\text{alg}})_i = 0, \quad \min_i (x_{\text{cvx}})_i = -1.152064525402398 \times 10^{-17}.$$

Moreover,

$$|\{i : (x_{\text{alg}})_i > 10^{-10}\}| = 3, \quad |\{i : (x_{\text{cvx}})_i > 10^{-10}\}| = 3.$$

Hence the custom algorithm and CVXPY return essentially the same solution and the same objective value. However, the custom algorithm is much faster:

$$\frac{\text{cvxpy_time}}{\text{algorithm_time}} \approx 34.99.$$

Therefore, for this special problem, the explicit simplex projection algorithm is preferable. It exploits the structure of the problem and achieves

$$O(n \log n)$$

complexity, while CVXPY incurs additional modeling and solver overhead.

■